Social Media Engagement Analysis

1. Project Objective

The objective of this project is to design and implement a relational database using MySQL to manage social media activities, including users, posts, followers, likes, comments, and notifications. It aims to analyze user engagement by retrieving post activities, calculating likes and comments, identifying active users, and examining follower relationships. The project also focuses on applying SQL concepts such as joins, aggregate functions, subqueries, stored procedures, user-defined functions, views, and triggers to efficiently manage and analyze social media data.

2. Problem Statement

- Design a relational database to organize social media users, posts, followers, likes, and comments.
- Retrieve user and post details using SQL queries and filtering conditions.
- Identify active users based on their post counts and follower activities.
- Analyze post engagement by calculating total likes and comments for each post.
- Evaluate user interactions by identifying users who liked or commented on posts.
- Implement functions, subqueries, stored procedures, views, and triggers to simplify database operations and automate notifications.

---

3. Attribute (Column /Features) Details:

Table 1: users

Attribute Name	Data Type	Description
userid	INT	Uniquely identifies each user (Primary Key).
username	VARCHAR(50)	Stores the user's username.
fullname	VARCHAR(100)	Stores the user's full name.
email	VARCHAR(100)	Stores the user's email address.
createdate	DATETIME	Stores the date and time when the user account was created.

Table 2: posts

Attribute Name	Data Type	Description
postid	INT	Uniquely identifies each post (Primary Key, Auto Increment).
userid	INT	Identifies the user who created the post (Foreign Key).
caption	TEXT	Stores the caption or text content of the post.
posted_at	DATETIME	Stores the date and time when the post was published (Default: Current Timestamp).

Table 3: followers

Attribute Name	Data Type	Description
followerid	INT	Identifies the user who follows another user (Foreign Key).
followingid	INT	Identifies the user being followed (Foreign Key).
follow_date	DATETIME	Stores the date and time when the user followed another user (Default: Current Timestamp).

Table 4: comments

Attribute Name	Data Type	Description
commentid	INT	Uniquely identifies each comment (Primary Key, Auto Increment).
postid	INT	Identifies the post on which the comment is made (Foreign Key).
userid	INT	Identifies the user who posted the comment (Foreign Key).
commenttext	VARCHAR(100)	Stores the text content of the comment.
commented_at	DATETIME	Stores the date and time when the comment was added (Default: Current Timestamp).

Table 5: likes

Attribute Name	Data Type	Description
likeid	INT	Uniquely identifies each like (Primary Key, Auto Increment).
postid	INT	Identifies the post that received the like (Foreign Key).
userid	INT	Identifies the user who liked the post (Foreign Key).
liked_at	DATETIME	Stores the date and time when the post was liked (Default: Current Timestamp).

---

4. Tools & Technologies

- **MySQL:** DDL & DML Commands, Primary Keys & Foreign Keys, MySQL Constraints, SELECT, WHERE, DISTINCT, IN, BETWEEN, NULL Handling, ORDER BY, LIMIT, GROUP BY, HAVING, Aggregate Functions, Joins, UNION, Built-in Functions, Subqueries, CTEs, User-Defined Functions (UDF), Stored Procedures, Views, Triggers.

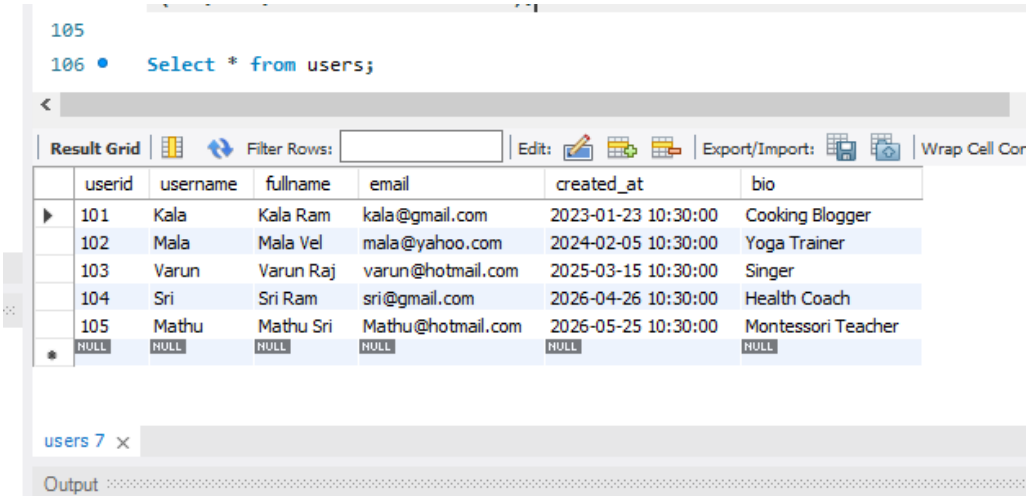
---

5. Data Pre-Processing (MySQL)

Tasks Performed:

1. Database and Table Creation (CREATE):

users table creation



posts table creation

```
107 •  Select * from posts;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

postid	userid	caption	posted_at
1	101	Creating memorable meals	2023-01-24 11:00:00
2	102	NULL	2024-02-06 11:00:00
3	103	Every song is a piece of my heart	2025-03-16 11:00:00
4	104	Healthy habits, happy life	2026-04-27 11:00:00
5	105	NULL	2026-05-26 11:00:00
NULL	NULL	NULL	NULL

posts 8 ×

Output

followers table creation

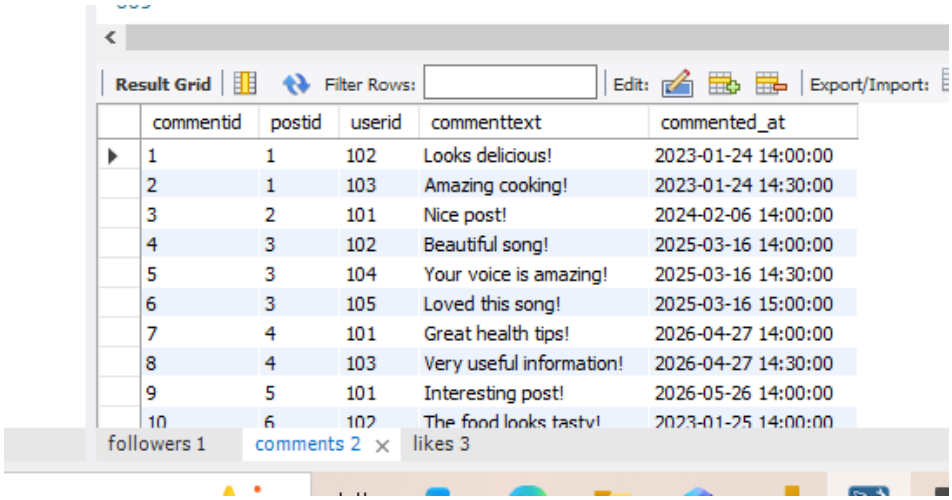
```
602
```

Result Grid | Filter Rows: | Export:

followerid	followingid	follow_date
101	102	2026-10-30 19:30:20
102	105	2026-10-30 19:30:20
103	101	2026-10-30 19:30:20
105	101	2026-10-30 19:30:20
104	103	2026-10-30 19:30:20

followers 1 × | comments 2 | likes 3

comments table creation

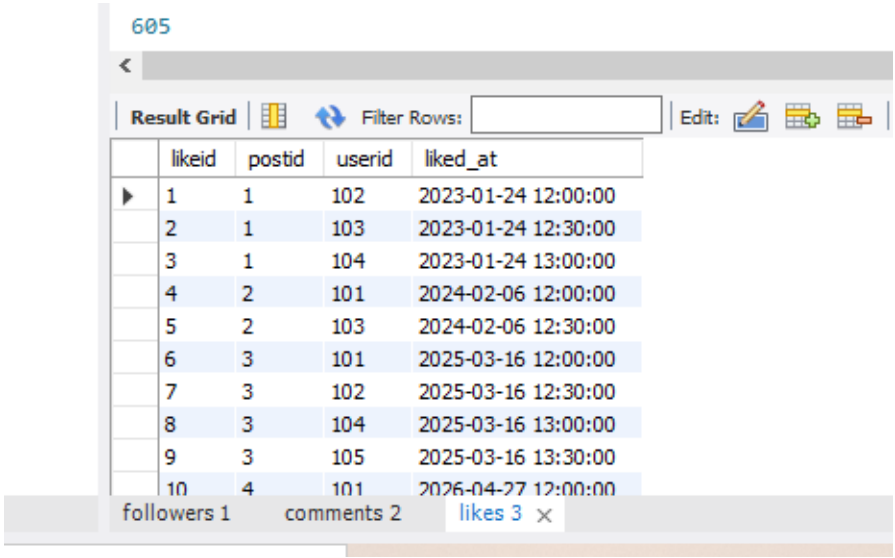


Result Grid | Filter Rows: | Edit: | Export/Import:

commentid	postid	userid	commenttext	commented_at
1	1	102	Looks delicious!	2023-01-24 14:00:00
2	1	103	Amazing cooking!	2023-01-24 14:30:00
3	2	101	Nice post!	2024-02-06 14:00:00
4	3	102	Beautiful song!	2025-03-16 14:00:00
5	3	104	Your voice is amazing!	2025-03-16 14:30:00
6	3	105	Loved this song!	2025-03-16 15:00:00
7	4	101	Great health tips!	2026-04-27 14:00:00
8	4	103	Very useful information!	2026-04-27 14:30:00
9	5	101	Interesting post!	2026-05-26 14:00:00
10	6	102	The food looks tasty!	2023-01-25 14:00:00

followers 1 | comments 2 | likes 3

likes table creation



605

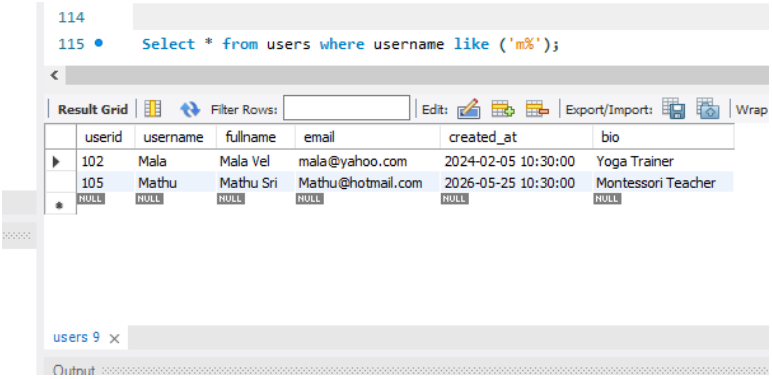
Result Grid | Filter Rows: | Edit:

likeid	postid	userid	liked_at
1	1	102	2023-01-24 12:00:00
2	1	103	2023-01-24 12:30:00
3	1	104	2023-01-24 13:00:00
4	2	101	2024-02-06 12:00:00
5	2	103	2024-02-06 12:30:00
6	3	101	2025-03-16 12:00:00
7	3	102	2025-03-16 12:30:00
8	3	104	2025-03-16 13:00:00
9	3	105	2025-03-16 13:30:00
10	4	101	2026-04-27 12:00:00

followers 1 | comments 2 | likes 3

2. DISTINCT, WHERE, IN, BETWEEN, NULL checks

- **Retrieve all users whose username starts with “m”**



```
114
115 •  Select * from users where username like ('m%');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

userid	username	fullname	email	created_at	bio
102	Mala	Mala Vel	mala@yahoo.com	2024-02-05 10:30:00	Yoga Trainer
105	Mathu	Mathu Sri	Mathu@hotmail.com	2026-05-25 10:30:00	Montessori Teacher
NULL	NULL	NULL	NULL	NULL	NULL

users 9

Output

- **Get posts posted BETWEEN '2023-01-01' AND '2025-12-31'**

```
119
120 •     Select * from posts where posted_at between '2023-01-01'
```

Result Grid | Filter Rows: | Edit: | Export/Import

postid	userid	caption	posted_at
1	101	Creating memorable meals	2023-01-24 11:00:00
2	102	NULL	2024-02-06 11:00:00
3	103	Every song is a piece of my heart	2025-03-16 11:00:00
NULL	NULL	NULL	NULL

posts 10 ×

- **Find users whose email domain is in: (@gmail.com, @hotmail.com, @yahoo.com)**

```
134         or email like '%@yahoo.com';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

userid	username	fullname	email	created_at	bio
101	Kala	Kala Ram	kala@gmail.com	2023-01-23 10:30:00	Cooking Blogger
102	Mala	Mala Vel	mala@yahoo.com	2024-02-05 10:30:00	Yoga Trainer
103	Varun	Varun Raj	varun@hotmail.com	2025-03-15 10:30:00	Singer
104	Sri	Sri Ram	sri@gmail.com	2026-04-26 10:30:00	Health Coach
105	Mathu	Mathu Sri	Mathu@hotmail.com	2026-05-25 10:30:00	Montessori Teacher
NULL	NULL	NULL	NULL	NULL	NULL

users 12 ×

3. ORDER BY, GROUP BY, LIMIT, HAVING:

- **Sort users by number of posts (DESC) and LIMIT 5.**

```
147
148 • select
149   userid,
150   count(postid) as Total_Posts
151   from posts
152   group by userid
153   order by Total_Posts DESC
154   Limit 5;
155
```

Result Grid | Filter Rows: | Export: | Wra

userid	Total_Posts
101	1
102	1
103	1
104	1
105	1

4. JOINS, UNION, BUxILT-IN FUNCTIONS

- **JOIN Report and Display postid, username, caption, total likes, total comments.**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

postid	username	caption	Total_likes	Total_comments
1	Kala	Creating memorable meals	3	2
2	Mala	NULL	2	1
3	Varun	Every song is a piece of my heart	4	3
4	Sri	Healthy habits, happy life	2	2
5	Mathu	NULL	1	1
6	Kala	deleciuos dinner	2	2
7	Varun	Melody hits	3	2
8	Varun	AR Rahman hits	3	3

- **UNION: Combine list of users who commented OR liked**

```
357     JOIN comments c
358         ON u.userid = c.userid
```

Result Grid | Filter Rows: | Export:

userid	username
101	Kala
102	Mala
103	Varun
104	Sri
105	Mathu

5. FUNCTIONS, SUBQUERIES, STORED PROCEDURES

- **Create a function: GetUserEngagement(user_id) and Returns total likes + comments made by the user**

```
399
400 •   -- To call
401
402     select userid, username, GetUserEngagement (userid) as total_likedandcommented_users  from users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

userid	username	total_likedandcommented_users
101	Kala	9
102	Mala	9
103	Varun	6
104	Sri	6
105	Mathu	6

Result 25 ×

- **Stored procedure to retrieve all posts for a given username.**

```
489
490 •   -- To call
491
492     call GetPostsbyusername('kala');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

postid	caption	username
1	Creating memorable meals	Kala
6	deleciuos dinner	Kala

Result 27 ×

Output

6. VIEWS & TRIGGERS

- Create a view for postsummary showing:

Postid, username, caption, total likes, total comments

```
551
552 •   select * from postsummary2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

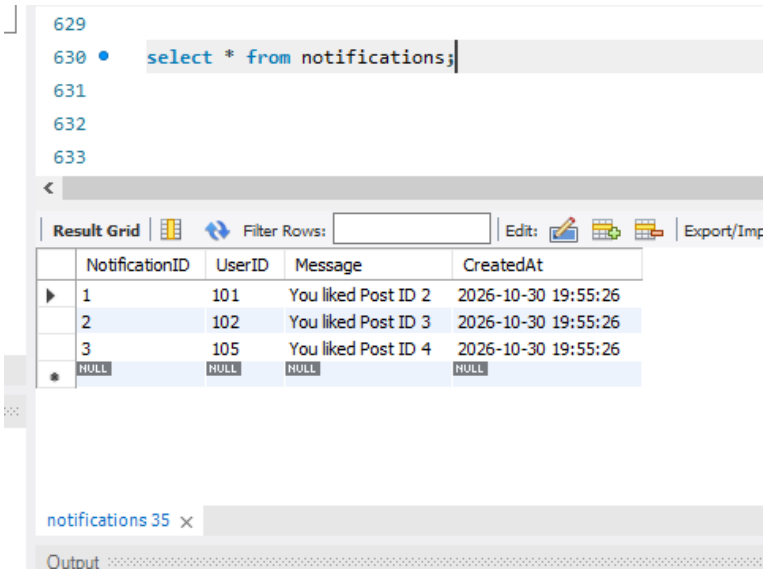
PostID	UserName	Caption	Total_Likes	Total_Comments
1	Kala	Creating memorable meals	3	2
2	Mala	NULL	2	1
3	Varun	Every song is a piece of my heart	4	3
4	Sri	Healthy habits, happy life	2	2
5	Mathu	NULL	1	1
6	Kala	deleciuos dinner	2	2
7	Varun	Melody hits	3	2
8	Varun	AR Rahman hits	3	3

postsummary2 32 ×

Output

- Trigger on likes table: When a user likes a post, insert a row into a new table:

notifications (userid, message, createdat)



---

7. Key Insights:

This project analyzes social media engagement using MySQL based on sample data containing **5 users**, **8 posts, 20 likes, 16 comments, and 5 follower** relationships. Descriptive Analysis: Identified **Varun** as the most active user with 3 posts, followed by **Kala** with 2 posts. Analyzed individual post engagement: Post ID 1 – "**Creating memorable meals**" received 3 likes and 2 comments (5 interactions); Post ID 2 – No caption received 2 likes and 1 comment (3 interactions); Post ID 3 – "**Every song is a piece of my heart**" received 4 likes and 3 comments (7 interactions); Post ID 4 – "**Healthy habits, happy life**" received 2 likes and 2 comments (4 interactions); Post ID 5 – No caption received 1 like and 1 comment (2 interactions); Post ID 6 – "**deleciuos dinner**" received 2 likes and 2 comments (4 interactions); Post ID 7 – "**Melody hits**" received 3 likes and 2 comments (5 interactions); and Post ID 8 – "**AR Rahman hits**" received 3 likes and 3 comments (6 interactions). Diagnostic Analysis: Identified that Post ID 3 generated the highest engagement with 7 interactions, followed by Post ID 8 with 6 interactions, whereas Post ID 5 received the lowest engagement with 2 interactions. This indicates variations in user engagement across different posts. Predictive Analysis: Posts receiving higher likes and comments may have the potential to attract more interactions in the future, while posts with lower engagement may require improvements in content. Prescriptive Analysis: Encourage users to publish engaging and relevant content regularly, improve interaction through likes and comments, and focus on improving posts with lower engagement. Monitor active users and follower activities to increase overall user participation and social media engagement.

8. Conclusions

This project is used to manage and analyze social media activities, including users, posts, followers, likes, and comments using MySQL. It helps identify active users, evaluate post-level engagement, and

understand user interaction patterns. The analysis identified Post ID 3 – "**Every song is a piece of my heart**" as the **highest-engagement post** with 4 likes and 3 comments (7 interactions), followed by Post ID 8 – "**AR Rahman hits**" with 3 likes and 3 comments (6 interactions). Post ID 5 – **No caption received** the **lowest** engagement with 1 like and 1 comment (2 interactions).

This project is used to manage and analyze social media activities, including user interactions, posts, followers, likes, and comments. It helps identify active users, track post engagement, understand follower relationships, and retrieve meaningful information from the database. By implementing MySQL queries, joins, functions, stored procedures, views, and triggers, the project demonstrates how SQL can be used to efficiently manage social media data and automate database operations.